Phase-field fracture models and MSPIN

Conor McCoid Blaise Bourdin

June 29, 2026

Abstract

—

1 Introduction

[What is brittle fracture]
[What is phase-field model]
[How has phase-field been computed in the past, with what limitations]
[Recent advances]
[Current proposition]
[Outline of paper]

2 Background

[Mathematical description of phase-field]
The phase-field energy may be expressed as

$$E_\ell(\boldsymbol{u},\boldsymbol{v}) = \int_\Omega \text{tensors} d\Omega + \int_{\partial\Omega} \left(\frac{\omega(\boldsymbol{v})}{4\ell} + \ell \left|\boldsymbol{v}\right|^2 \right) dS. \tag{1}$$

[Irreversibility condition]

2.1 Alternate Minimization

[AltMin, using notation from mathematical description of phase-field]

AltMin may be viewed as a field-split multiplicative Schwarz method. The two 'fields' of damage and displacement are treated separately and sequentially. This allows the use of some of the techniques reserved for Schwarz methods to be applied here. In particular, in Section 2.3 we will examine applying MSPIN to the problem.

2.2 Active set methods

2.3 MSPIN

Multiplicative Schwarz preconditioning inexact Newton (MSPIN) was first described in (nb: cite Cai and Keyes), with further explanation for field-split Schwarz methods in (nb: cite Liu and Keyes). MSPIN has been previously applied to phase-field fracture models in (nb: cite 3K). There, the authors used cubic backtracking to globalize the Newton direction, see Section 2.4 below. Note that their model differs slightly from the one presented here.

We present MSPIN as applied to AltMin. [fill in once notation is specified from prev. section]

2.4 Cubic Backtracking

Many applications of Newton's method require globalization techniques to achieve convergence when starting away from the minimum. When the Newton direction overshoots the minimum, these globalization techniques should search for a minimum between the starting point and the destination, usually through a line search.

Cubic backtracking (nb: cite) performs this line search by repeatedly interpolating cubic polynomials and finding their minima. Note that a cubic polynomial interpolant requires four pieces of information, but Newton's method only provides three: the value of the function at the starting point, $f(\boldsymbol{x})$; the value of the function in the Newton direction, $f(\boldsymbol{x} + \boldsymbol{p})$, and; the derivative of the function in the Newton direction, $\boldsymbol{p} \cdot \nabla f(\boldsymbol{x})$.

To get a fourth piece of information, first form a quadratic interpolant with this information. Find the minimum of this polynomial and test the value of the function there, thus getting enough information for a cubic polynomial. If the function is smaller at this point, then this ends the line search. Otherwise, find the minimum of the cubic interpolant and repeat. This algorithm is summarized in Algorithm 1.

3 Interpolant minimization

When using MSPIN, one has two potential directions for minimization: the AltMin direction $\boldsymbol{q}$ and the Newton direction $\boldsymbol{p}$. Let's try using both while applying similar ideas to cubic backtracking. Rather than find the minimum of a single variable cubic polynomial, find the minimum of a multivariate quadratic polynomial interpolant. Here we present three options for interpolants to minimize.

3.1 Parallelogram

We begin by considering a 2D interpolant using the AltMin and Newton directions, $\boldsymbol{q}$ and $\boldsymbol{p}$. For this interpolant, we need six pieces of information. Five are provided by MSPIN: the value of the function at the starting point, $f(\boldsymbol{x})$; the

Algorithm 1 Cubic backtracking (nb: cite)
p: search direction, x: current position, f: objective function, α: bound on step size, τ: tolerance (nb: of some kind, check)

Calculate $c = p^\top \nabla f(x)$
if $f(x+p) \leq f(x) + \alpha c$ **then**
    Set $\lambda = 1$
else if $\|p\| \leq \tau$ **then**
    (nb: check)
else
    Find λ such that $x + \lambda p$ minimizes the degree 2 polynomial interpolating $f(x)$, $f(x+p)$, and g:

$$\lambda = -c / \left(2\left(f(x+p) - f(x) - c\right)\right)$$

    Set $\lambda_0 = 1$, $\lambda_1 = \lambda$
    while $f(x + \lambda p) > f(x) + \alpha \lambda c$ **do**
        Find the coefficients of the degree 3 polynomial interpolating $f(x)$, $f(x + \lambda_0 p)$, $f(x + \lambda_1 p)$, and g:

$$\begin{bmatrix} a \\ b \end{bmatrix} = \frac{1}{\lambda_1 - \lambda_0} \begin{bmatrix} 1/\lambda_1^2 & -1/\lambda_0^2 \\ -\lambda_0/\lambda_1^2 & \lambda_1/\lambda_0^2 \end{bmatrix} \begin{bmatrix} f(x+\lambda_1 p) - f(x) - \lambda_1 c \\ f(x+\lambda_0 p) - f(x) - \lambda_0 c \end{bmatrix}$$

        if $a = 0$ **then**
            $\lambda = -c/2b$
        else
            $\lambda = \frac{-b + \sqrt{b^2 - 3ac}}{3a}$
        end if
        if $\lambda > 0.5\lambda_1$ **then**
            $\lambda = 0.5\lambda_1$
        end if
        $\lambda_0 = \lambda_1$, $\lambda_1 = \lambda$
    end while
end if
Return λ, amount to travel in search direction p to minimize $f(x)$

value of the function in the AltMin direction, $f(\boldsymbol{x}+\boldsymbol{q})$; the value of the function in the Newton direction, $f(\boldsymbol{x}+\boldsymbol{p})$; the derivative of the function in the AltMin direction, $\boldsymbol{q}\cdot\nabla f(\boldsymbol{x})$, and; the derivative of the function in the Newton direciton, $\boldsymbol{p}\cdot\nabla f(\boldsymbol{x})$.

One choice for a sixth piece of information is the value of the function at a point symmetric to the starting point. Suppose that the AltMin and Newton directions form two sides of a parallelogram. Then the other corners of the parallelogram are the starting point and $\boldsymbol{x}+\boldsymbol{p}+\boldsymbol{q}$. It is at this new point that we evaluate the function and find our sixth piece of information.

We seek a point $\boldsymbol{x}+\alpha\boldsymbol{q}+\beta\boldsymbol{p}$ that minimizes the objective function. The interpolant appears as $P(\alpha,\beta)=a\alpha^2+b\alpha\beta+c\beta^2+d\alpha+e\beta+f$. It is straightforward to show how our six pieces of information relate to our six coefficients:

$$\begin{aligned} d&=\boldsymbol{q}\cdot\nabla f(\boldsymbol{x}), \qquad e=\boldsymbol{p}\cdot\nabla f(\boldsymbol{x}), \qquad f=f(\boldsymbol{x}),\\ a&=f(\boldsymbol{x}+\boldsymbol{q})-d-f, \qquad c=f(\boldsymbol{x}+\boldsymbol{p})-e-f,\\ b&=f(\boldsymbol{x}+\boldsymbol{q}+\boldsymbol{p})+f-f(\boldsymbol{x}+\boldsymbol{q})-f(\boldsymbol{x}+\boldsymbol{p}). \end{aligned}$$

The minimum of this interpolant occurs at

$$\alpha=\frac{be-2cd}{4ac-b^2}, \quad \beta=\frac{-db-2ae}{4ac-b^2}.$$

It's possible for the minimum of the polynomial to occur far outside of the parallelogram. This may be acceptable, but could also destabilize convergence. To stay within the parallelogram, we can also check the minima on the boundary:

$$(0,-e/2c), \qquad (-d/2a,0), \qquad (1,-(e+b)/2c), \qquad (-(d+b)/2c,1).$$

In practice, it is advisable to have β, the amount to step in the Newton direction, be between -1 and 1, while α, the amount to step in the AltMin direction, can be any positive value. We will see examples of this behaviour in the next section.

Before accepting the critical point of the interpolant as the next step in the algorithm, one should perform the second derivative test to confirm that this point is in fact a minimum. The determinant of the Jacobian matrix is $D=4ac-b^2$, used in the denominator of both α and β. If $D>0$ and $a>0$, then the critical point is indeed a minimum. Otherwise, this point may be a maximum or a saddle point. This becomes more likely when the calculation of the gradient of the function is subject to numerical error, especially when the magnitude of the gradient is small in the AltMin and Newton directions.

Should the calculated critical point be deemed unacceptable, either by being too far from the starting point or failing the second derivative test, then one should change strategies. One can choose one of the other interpolants suggested here, or select one of the three corners of the parallelogram with the lowest energy as the next step.

3.2 Triangle

If one wants to form interpolants using only evaluations of the objective function and not its gradient, then one can form a P2 finite element by evaluating the

objective function at six points. Four of these points are the corners of the parallelogram described previously. The remaining two are $\boldsymbol{x}+2\boldsymbol{p}$ and $\boldsymbol{x}+2\boldsymbol{q}$. The coefficients of the polynomial $P(\alpha,\beta)$ are then:

$$\begin{aligned} f &= f(\boldsymbol{x}), \qquad b = f(\boldsymbol{x}+\boldsymbol{q}+\boldsymbol{p}) + f - f(\boldsymbol{x}+\boldsymbol{q}) - f(\boldsymbol{x}+\boldsymbol{p}), \\ a &= \frac{1}{2}\left(f(\boldsymbol{x}+2\boldsymbol{q}) + f\right) - f(\boldsymbol{x}+\boldsymbol{q}), \qquad d = f(\boldsymbol{x}+\boldsymbol{q}) - f - a, \\ c &= \frac{1}{2}\left(f(\boldsymbol{x}+2\boldsymbol{p}) + f\right) - f(\boldsymbol{x}+\boldsymbol{p}), \qquad e = f(\boldsymbol{x}+\boldsymbol{p}) - f - c. \end{aligned}$$

Like the parallelogram, the minimum of this interpolant occurs at

$$\alpha = \frac{be-2cd}{4ac-b^2}, \quad \beta = \frac{-db-2ae}{4ac-b^2}.$$

If one started with the parallelogram interpolant but were unsatisfied by its critical point, then the triangle interpolant requires only two more evaluations of the objective function. If the new interpolant still fails the second derivative test, then the next step in the algorithm can be chosen as that which has the lowest value of the objective function from amongst the five values already found.

3.3 Tetrahedron

Lastly, let us consider forming a 3D interpolant by including the intermediate step in AltMin. That is, let $\boldsymbol{p}$ and $\boldsymbol{q}$ remain the same, and add in the direction $\boldsymbol{r}$, which is the restriction of $\boldsymbol{q}$ to the damage degrees of freedom.

To form the P2 finite element for this 3D interpolant, we need 10 evaluations of the objective function. This will be the six already used for the triangle interpolant, as well as $f(\boldsymbol{x}+\boldsymbol{r})$, $f(\boldsymbol{x}+\boldsymbol{r}+\boldsymbol{q})$, $f(\boldsymbol{x}+\boldsymbol{r}+\boldsymbol{p})$, and $f(\boldsymbol{x}+2\boldsymbol{r})$. The interpolant now appears as

$$Q(\alpha,\beta,\gamma) = a\alpha^2 + b\beta^2 + c\gamma^2 + d\alpha\beta + e\beta\gamma + f\alpha\gamma + g\alpha + h\beta + i\gamma + j.$$

These coefficients are equal to

$$\begin{aligned} j &= f(\boldsymbol{x}), \qquad d = f(\boldsymbol{x}+\boldsymbol{p}+\boldsymbol{q}) - f(\boldsymbol{x}+\boldsymbol{q}) - f(\boldsymbol{x}+\boldsymbol{p}) + j, \\ &\qquad\quad e = f(\boldsymbol{x}+\boldsymbol{r}+\boldsymbol{p}) - f(\boldsymbol{x}+\boldsymbol{p}) - f(\boldsymbol{x}+\boldsymbol{r}) + j, \\ &\qquad\quad f = f(\boldsymbol{x}+\boldsymbol{q}+\boldsymbol{r}) - f(\boldsymbol{x}+\boldsymbol{r}) - f(\boldsymbol{x}+\boldsymbol{q}) + j, \\ a &= \frac{1}{2}\left(f(\boldsymbol{x}+2\boldsymbol{q}) - 2f(\boldsymbol{x}+\boldsymbol{q}) + j\right), \qquad g = f(\boldsymbol{x}+\boldsymbol{q}) - j - a, \\ b &= \frac{1}{2}\left(f(\boldsymbol{x}+2\boldsymbol{p}) - 2f(\boldsymbol{x}+\boldsymbol{p}) + j\right), \qquad h = f(\boldsymbol{x}+\boldsymbol{p}) - j - b, \\ c &= \frac{1}{2}\left(f(\boldsymbol{x}+2\boldsymbol{r}) - 2f(\boldsymbol{x}+\boldsymbol{r}) + j\right), \qquad i = f(\boldsymbol{x}+\boldsymbol{r}) - j - c. \end{aligned}$$

The minimum occurs at the solution to the following system:

$$\begin{bmatrix} 2a & d & f \\ d & 2b & e \\ f & e & 2c \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix} = - \begin{bmatrix} g \\ h \\ i \end{bmatrix},$$

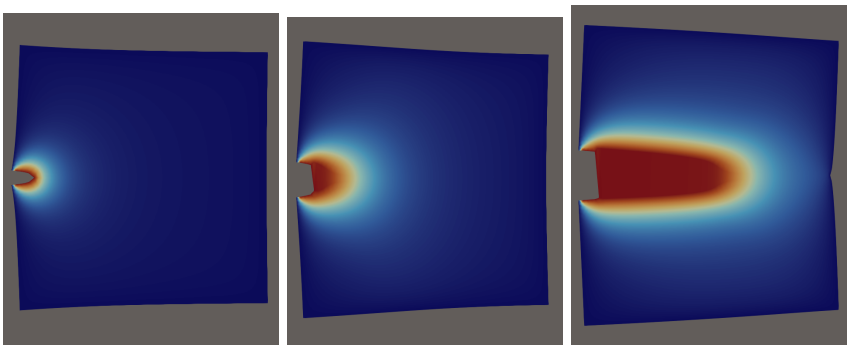

Figure 1: Crack progression of the surfing boundary conditions example.

but only if the matrix of this system is positive definite. Otherwise, this critical point is a saddle point or a maximum.

4 Numerical comparisons

Numerical experiments conducted for this article are run using FEniCSx for its finite element library, petsc4py for solving linear systems, and mpi4py for parallelization. The Docker image used for these experiments can be found at (nb: DockerHub link) (nb: cite Docker and Apptainer). Code for these experiments can be found at (nb: GitHub link).

4.1 Surfing boundary conditions

For this example, we begin with an initial crack of length (nb: length) in an otherwise rectangular domain with an unstructured grid of (nb: number) points. The boundary conditions set the damage at maximum on the initial crack and zero on the boundary. For displacement, the boundary conditions are of Neumann type, setting

$$\frac{\partial u}{\partial x} = \frac{K}{2\mu}\sqrt{\frac{r(t)}{2\pi}}\,(k - \cos\theta(t))\cos\frac{\theta(t)}{2}, \quad \frac{\partial u}{\partial y} = \frac{K}{2\mu}\sqrt{\frac{r(t)}{2\pi}}\,(k - \cos\theta(t))\sin\frac{\theta(t)}{2},$$
$$r(t) = \sqrt{(x-t)^2 + y^2}, \quad \theta(t) = \arctan\frac{y}{x-t},$$

where t defines the force exerted on the system, (nb: and define the remaining parameters). Figure 1 shows the progression of the crack for this example.

Figure 2 shows the resulting energies produced by each of the five methods. This can be used to determine correctness of the solutions. We see that the elastic energy for the cubic backtracking continues to rise over all time steps, while for all other methods there is a point where it decreases. Ultimately, the other four methods find lower total energy, and therefore produce more correct crack progression.

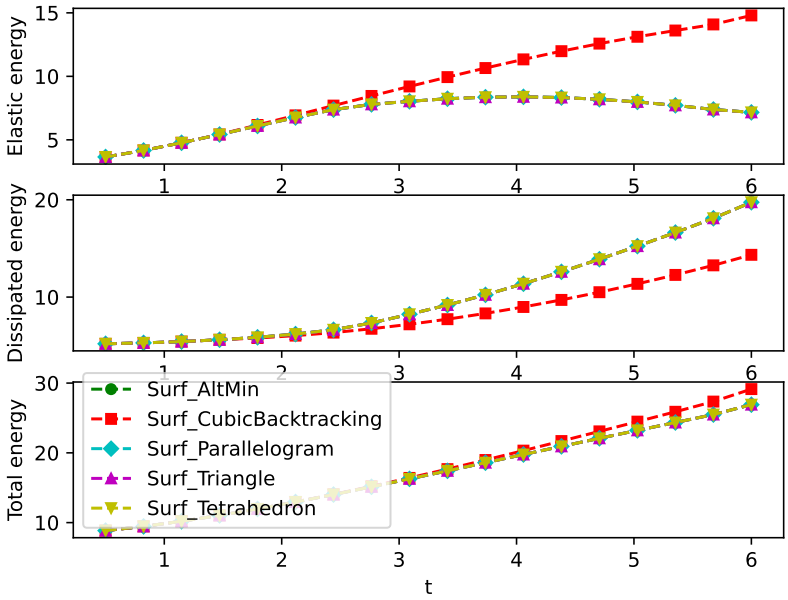

Figure 2: Energy for the surfing example using the five methods tested.

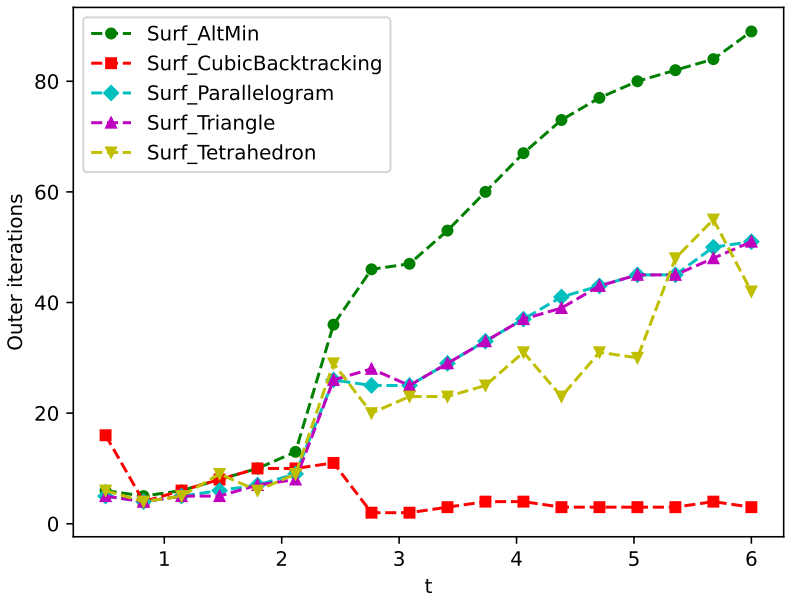

Figure 3: Outer iterations (number of linear solves of the AltMin system) for each load step of the surfing example.

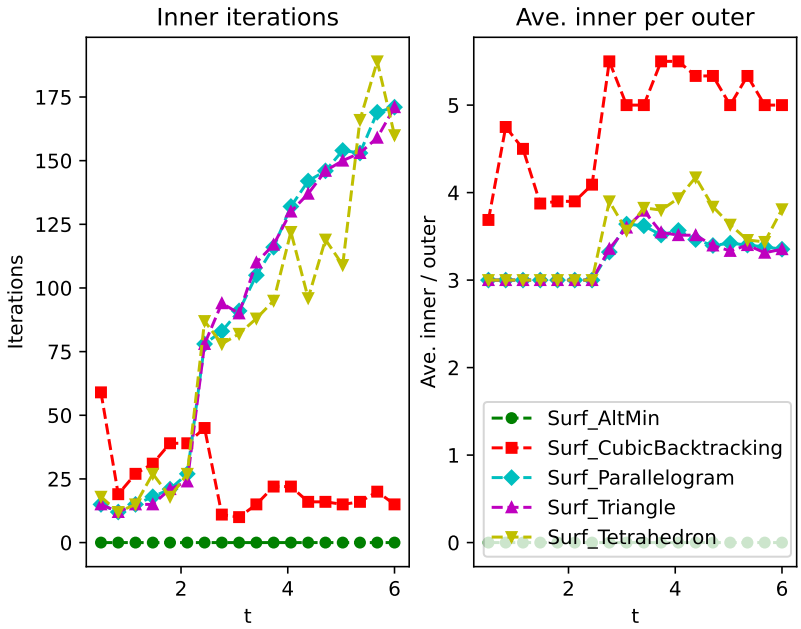

Figure 4: On the left, total number of inner iterations (matrix-vector multiplications required to solve the Newton system to within tolerance) for the four methods that use inexact Newton. On the right, average number of inner iterations per outer iterations.

Figure 3 gives the number of outer iterations required for each load step to be completed for each of the five methods. An outer iteration is characterized by a solve of the linear system defined by AltMin. Cubic backtracking requires the lowest number of outer iterations, but we have seen that it produces the most problematic solution. The three interpolant techniques require roughly half the number of outer iterations of AltMin. The parallelogram and triangle interpolants behave about the same, while the tetrahedron performs marginally better.

Computation time is determined not only by the number of outer iterations, but also the number of inner iterations, a measure of the time to solve the inexact Newton step, see Figure 4. However, the average number of inner iterations per outer iteration remains fairly constant for each method, meaning reducing the number of outer iterations remains the goal. Cubic backtracking has a high average, at times over 5, while the three types of interpolants all remain between 3 and 4.

Wall clock time includes several steps that can be avoided through code design, such as print statements and communication. However, it gives some

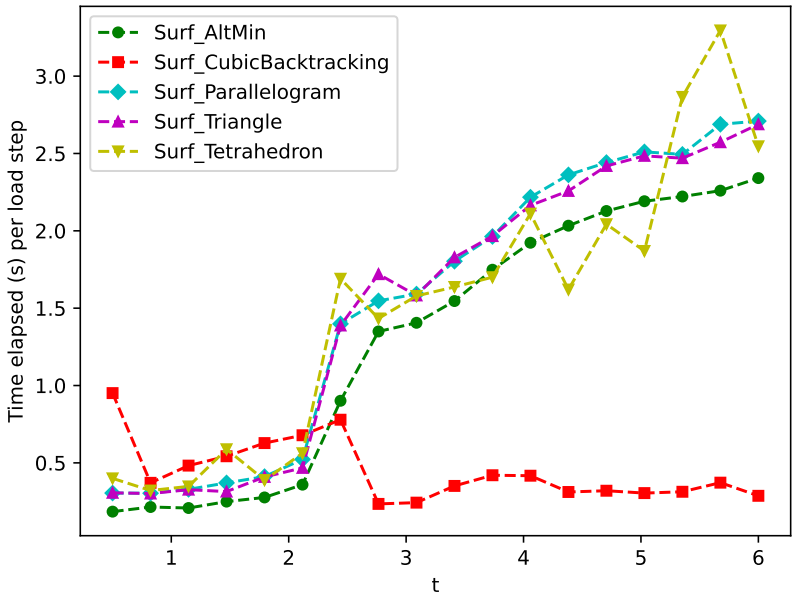

Figure 5: Wall clock time for solving the surfing example for all five methods.

Method	Flops	% AltMin	% MSPIN
AltMin	3.970×10^{10}	99.9	
Cubic backtracking	1.538×10^{10}	43.6	56.3
Parallelogram	5.178×10^{10}	45.9	53.7
Triangle	5.175×10^{10}	45.8	53.9
Tetrahedron	4.751×10^{10}	44.1	55.5

Table 1: Flop count for the surfing example.

Method	Time (s)	% AltMin	% MSPIN
AltMin	3.583×10^{1}	64.3	
Cubic backtracking	1.946×10^{1}	22.6	14.9
Parallelogram	3.963×10^{1}	35.2	23.4
Triangle	3.905×10^{1}	35.9	23.8
Tetrahedron	3.842×10^{1}	32.7	22.6

Table 2: Timing for the surfing example.

indication of the effectiveness of theoretical speedups. Figure 5 gives the wall clock time for each of the five methods for each load step. We see that AltMin has the best time of the accurate methods even though it requires the most number of outer iterations. It is clear that care must be taken to reduce the number of flops spent on the inexact Newton step.

Table 1 shows the number of flops for all five methods tested for the surfing example, while Table 2 gives the wall clock time for each. Also shown is the percentage of flops and time spent completing the main computations: solving the linear system to compute the AltMin direction, and approximately solving the nonlinear system to compute the MSPIN direction. These flop counts and timings are found using PETSc's built-in log view, and may be subject to errors.

While the MSPIN direction takes more flops to compute, it takes less time, likely due to better parallelization and preconditioning. Since much of the MSPIN system has already been factorized during the solve of the AltMin direction, this factorization speeds up subsequent calculations.

We also note that approximately 40% of time is spent on other tasks, though these tasks account for almost no flops. This suggests communication issues arising from the parallelization, or wayward print statements.

4.2 Crack-tip force method (CTFM)

[introduce method and equations]

[figure showing 3 hole domain]

Figure 6 shows the resulting energies produced by the five methods. As in the surfing example, the cubic backtracking is reticent to allow cracks to grow, instead preferring to increase elastic energy. As the other four methods all have lower total energy potentials, their solutions indicate more appropriate crack progressions.

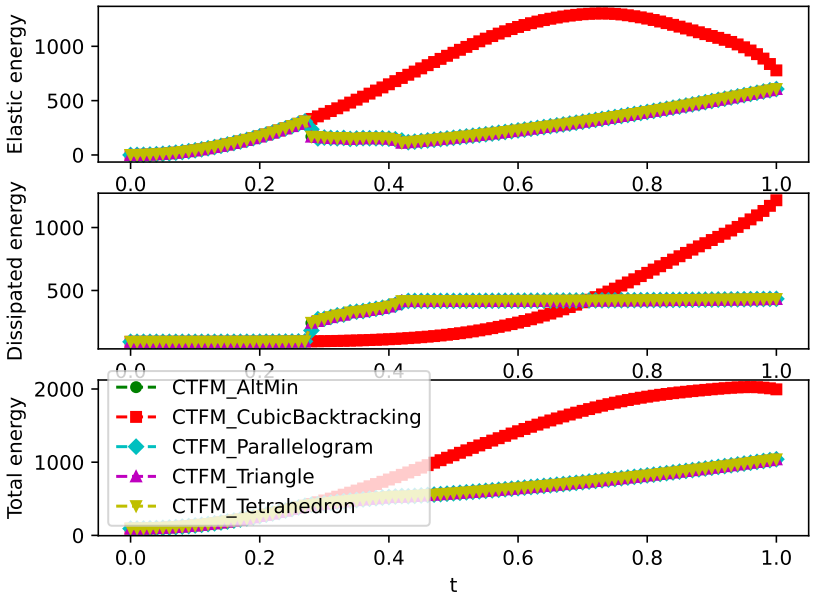

Figure 6: Energy for the CTFM example using the five methods tested.

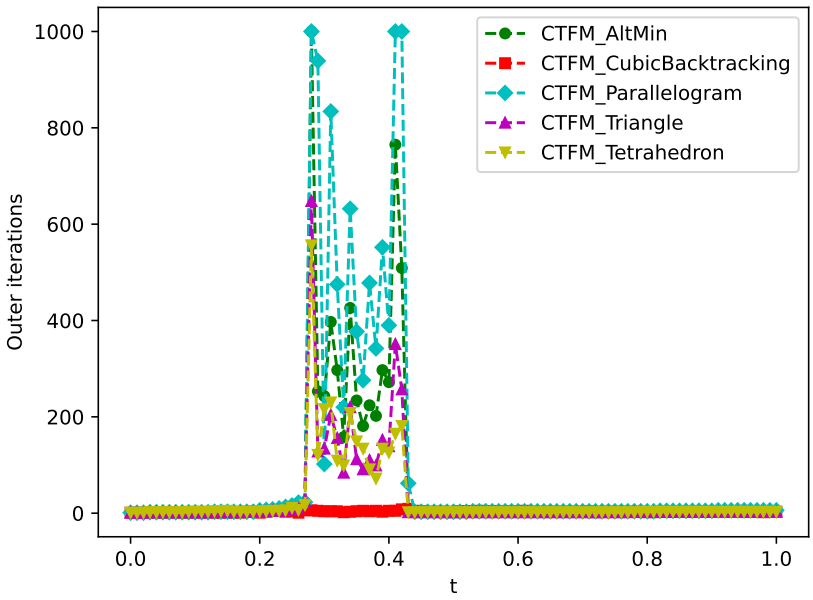

Figure 7: Outer iterations (number of linear solves of the AltMin system) for each load step of the CTFM example.

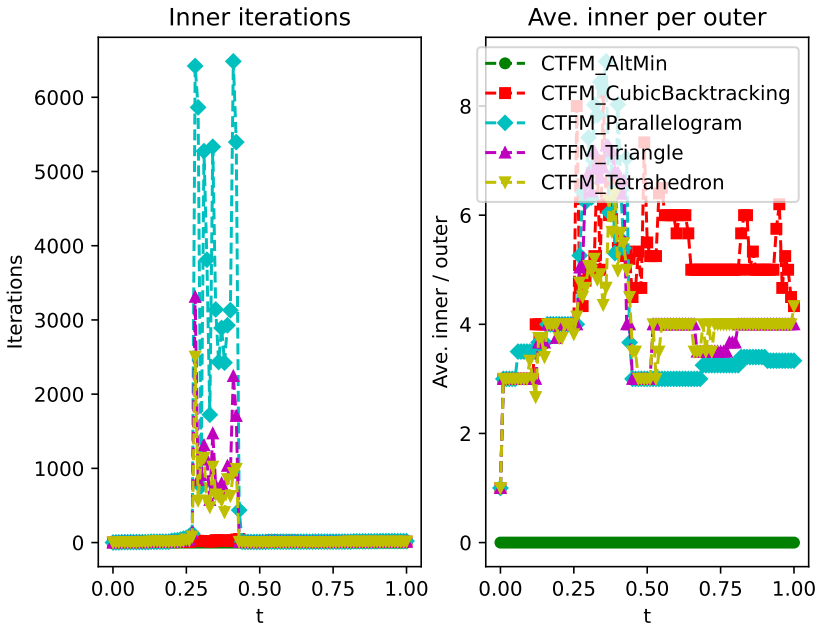

Figure 8: On the left, total number of inner iterations for the four non-AltMin methods. On the right, average number of inner iterations per outer iteration.

Figure 7 gives the number of outer iterations required for the completion of each load step. Each outer iteration represents the process of solving the linear system of AltMin. Cubic backtracking again requires the lowest number of outer iterations, but at the expense of the highest potential energy.

The number of outer iterations only becomes significant for the load steps where the crack progresses rapidly. In this regime, there are many ways for the crack to propagate that will result in lower energy, meaning the energy landscape has many saddle points and local minima. Both the triangle and tetrahedron interpolants consistently have fewer outer iterations than AltMin. The parallelogram interpolant is also often competitive, but not in a predictable way.

Figure 8 shows the additional cost of the MSPIN-based methods over AltMin: the number of matrix-vector multiplications required to solve the inexact Newton problem. The number of inner iterations is closely correlated with the number of outer iterations performed. However, the average number of inner iterations per outer iteration spikes during the crack propagation regime, suggesting the MSPIN system becomes more complicated to resolve at this stage.

Figure 9 gives a measure of the amount of time each method takes to complete each load step. As expected, most time is spent during the crack propaga-

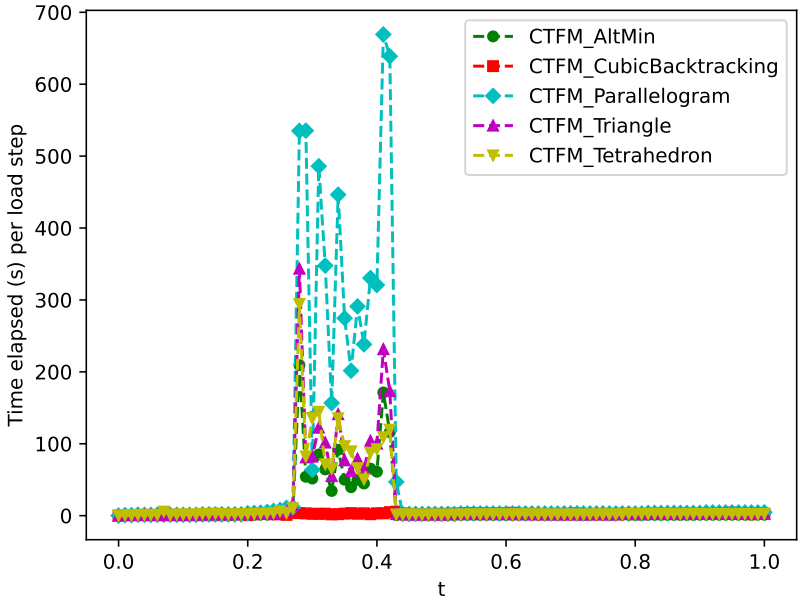

Figure 9: Wall clock time for each load step for the CTFM example.

Method	Flops	% AltMin	% MSPIN
AltMin	1.160×10^{14}	100	
Cubic backtracking	2.196×10^{13}	36.8	63.1
Parallelogram	5.631×10^{14}	32.4	67.6
Triangle	1.744×10^{14}	36.6	63.4
Tetrahedron	1.670×10^{14}	35.0	65.0

Table 3: Flop count for the CTFM example.

Method	Time (s)	% AtlMin	% MSPIN
AltMin	1.321×10^{3}	96.7	
Cubic backtracking	2.760×10^{2}	35.3	48.9
Parallelogram	5.893×10^{3}	33.7	64.2
Triangle	2.066×10^{3}	38.2	58.3
Tetrahedron	1.871×10^{3}	35.7	60.5

Table 4: Timings for the CTFM example.

tion regime. AltMin and the triangle and tetrahedron interpolants take roughly the same amount of time, with the parallelogram interpolant struggling due to its high number of outer iterations.

Table 3 gives the number of flops for each method on this example, while Table 4 gives the timings. Both tables include the percentage of each measure that is spent on the linear and nonlinear solves. The difference in costs between linear and nonlinear solves here is much more stark than in the previous example, with the nonlinear solves costing nearly twice as much as the linear solves in terms of both flops and time. This suggests a lack of scalability in the preconditioning methods used to speed up the MSPIN calculations.

It is clear that using MSPIN with the interpolant minimization techniques can reduce the number of outer iterations, often significantly. However, doing so greatly increases the computational cost of each outer iteration, up to 290%. To be competitive against AltMin, the number of outer iterations will need to be reduced by a factor of 3.